

Einführung in Manim

Einleitung

Manim bietet die Möglichkeit, mathematische Zusammenhänge zu animieren. Das wollen wir uns im nächsten Kapitel anschauen.

1 Wie sieht ein Manim-Programm aus

Beispiel:

```
from manim import * # Import von Manim

class Example(Scene):
    def construct(self):
        dot = Dot(color=BLUE)
        square = Square(color=GREEN, fill_opacity=0.5)

        dot.next_to(square, DOWN)
        self.add(square, dot)
```

Wenn wir Manim-Programme mit Jupyter oder Colab rendern wollen, müssen wir in die erste Zeile der Zelle folgendes schreiben:

```
%%manim -qm <Classname>
```

wobei <Classname> den Namen der jeweiligen Klasse beschreibt.

2 Mobjects

```
from manim import *

class Shapes(Scene):
    def construct(self):
```

```
circle = Circle()
square = Square()
triangle = Triangle()

circle.shift(LEFT)
square.shift(UP)
triangle.shift(RIGHT)

self.add(circle, square, triangle)
```

Mögliche Mobjects sind u.A.

- Polygon,
- Rectangle,
- RegularPolygon,
- Square
- Star,
- Triangle,
- Line,
- Angle bzw. RightAngle,
- Dot,
- Tex bzw. MathTex

Der Koordinatenursprung wird mit `ORIGIN` bezeichnet. Die Richtungen sind `UP`, `DOWN`, `LEFT` und `RIGHT`.

Methoden wie `.shift()` können auch zusammengefügt werden. So haben folgende beiden Programmausschnitte die gleiche Funktion:

```
square = Square()
square.shift(DOWN)

square = Square().shift(DOWN)
```

Andere Möglichkeiten zur Positionierung sind folgende:

```

from manim import *

class Shapes(Scene):
    def construct(self):
        circle = Circle()
        square = Square()
        triangle = Triangle()

        circle.move_to(2 * LEFT)
        square.next_to(circle, LEFT)
        triangle.align_to(circle, RIGHT)

        self.add(circle, square, triangle)

```

3 Styling

```

from manim import *

class MobjectStyling(Scene):
    def construct(self):
        circle = Circle().shift(LEFT)
        square = Square().shift(UP)
        triangle = Triangle().shift(RIGHT)

        circle.set_stroke(color=GREEN, width=20)
        square.set_fill(YELLOW, opacity=1.0)
        triangle.set_fill(PINK, opacity=0.5)

        self.add(circle, square, triangle)

```

Die Reihenfolge der Mobjects kann man durch die Reihenfolge in `.add` ändern:
`self.add(circle, square, triangle)` vs. `self.add(triangle, circle, square)`

4 Animationen

```

from manim import *

class SomeAnimations(Scene):
    def construct(self):
        square = Square()

```

```

self.play(FadeIn(square))

self.play(Rotate(square, PI/4))

self.play(FadeOut(square))

self.wait(1)

```

Wir können aber auch über die Methode `.animate` auf das Mobject zugreifen:

```

from manim import *

class SomeAnimations(Scene):
    def construct(self):
        square = Square()

        self.add(square)

        self.play(square.animate.set_fill(WHITE))

        self.play(square.animate.shift(UP).rotate(PI/4), run_time = 3)

        self.play(FadeOut(square))

        self.wait(1)

```

```

from manim import *

class SomeAnimations(Scene):
    def construct(self):
        tex = MathTex(r'\frac{10}{x}')

        self.add(tex)

        self.play(tex.animate.shift(UP).rotate(PI/4), run_time = 3)

        self.play(FadeOut(tex))

        self.wait(1)

```